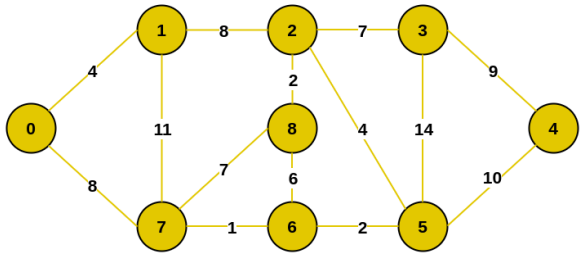
Minimum Spanning Tree Using MapReduce Framework

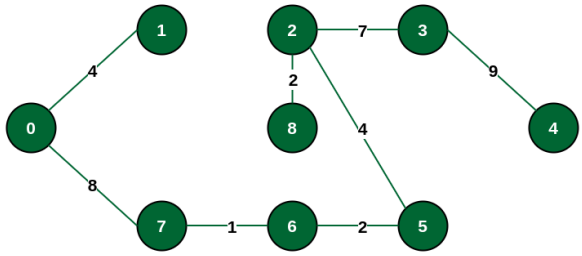
Team - 69 Bassam Adnan 2023121003 Rajendraprasad Saravanan 2023121004

1 Problem Statement

The Minimum Spanning Tree (MST) is a fundamental problem in graph theory, where the goal is to find a subset of edges that connects all vertices in a graph without cycles and with the minimum possible total edge weight. This project focuses on implementing Borůvka's algorithm to compute an MST using a distributed framework. The MapReduce paradigm was chosen for scalability, as it allows parallel processing of graph edges over large datasets distributed across multiple machines.



Example of a Graph



The final structure of MST

2 Literature Review

Several algorithms exist to solve the MST problem, such as Kruskal's and Prim's algorithms. However, Borůvka's algorithm is particularly suitable for parallel computation due to its ability to identify the smallest edge for each component independently, making it ideal for a MapReduce environment. The following references are particularly relevant in the context of this problem:

- **Borůvka, O. (1926).** *Jisté otázky minimální (Some questions on minimum).* This seminal work introduced Borůvka's algorithm, one of the earliest methods for finding a Minimum Spanning Tree (MST). Borůvka's method focuses on iteratively connecting components by selecting the smallest edge incident to each component. Its simplicity and parallelizable nature make it foundational for distributed implementations.
- **Gallager, R. G., Humblet, P. A., & Spira, P. M. (1983).** *A distributed algorithm for minimum-weight spanning trees.* This paper presents a distributed algorithm for constructing MSTs in networked

environments. It is a landmark study in distributed computing, offering insights into communication-efficient methods for solving the MST problem. The algorithm proposed is particularly influential in modern distributed systems and serves as a theoretical basis for scalable MST computation.

- **Dean, J., & Ghemawat, S. (2008).** *MapReduce: Simplified Data Processing on Large Clusters.* Dean and Ghemawat's work introduces the MapReduce paradigm, a framework for processing vast amounts of data in parallel across distributed systems. This paper highlights the scalability and fault tolerance of the MapReduce model, which is instrumental for implementing algorithms like Borůvka's in large-scale distributed settings.

3 Scope

The scope of this project involves implementing a generalized MapReduce module from scratch and adapting the parallel Borůvka's algorithm to it.

3.1 MapReduce Module

In this project, we aim to develop a Map Reduce library (paper Link) (or) a module in Go. MapReduce is a programming model and an associated implementation for processing and generating large data sets. It takes in a Map Function and a Reduce Function, the outputs of the map functions are passed to the reduce function (by taking the intermediate keys). These map/reduce functions are given by the user whereas the system itself is initialized with a pre-determined number of "workers" which will be responsible for executing the said functions on the data. Programs written in the map reduce style should be automatically parallelized and are typically run on a large cluster of machines with some form of inter-machine communication method (see Implementation Details section). The execution is controlled by a Master process which assigns tasks or jobs to Worker processes. Well known data-intensive algorithms can be efficiently processed using this model, to name a few- Distributed Grep, Inverted index, Count of URL Access frequency (to understand these please refer to the paper). The three phases involved can be summarized as follows

- **Map Phase:** The input data is first split into smaller blocks. Each block is then assigned to a mapper for processing. Each 'worker' node applies the map function to the local data, and writes the output to temporary storage. The primary (master) node ensures that only a single copy of the redundant input data is processed.
- **Shuffle, Combine, and Partition Phase:** Worker nodes redistribute data based on the output keys produced by the map function, such that all data belonging to one key is located on the same worker node. As an optional process, the combiner (a reducer) can run individually on each mapper server to reduce the data on each mapper even further, making reducing the data footprint and shuffling and sorting easier. Partition, which is not optional, is the process that decides how the data has to be presented to the reducer and also assigns it to a particular reducer.
- **Reduce Phase:** A reducer cannot start while a mapper is still in progress. Worker nodes process each group of (key,value) pairs output data, in parallel to produce (key,value) pairs as output. All the map output values that have the same key are assigned to a single reducer, which then aggregates the values for that key.

This can be summarized based on the equations given in the paper by Google:

$$\text{map} : (k_1, v_1) \rightarrow \text{list}(k_2, v_2)$$
$$\text{reduce} : (k_2, \text{list}(v_2)) \rightarrow \text{list}(v_2)$$

3.2 Adapting Borůvka's Algorithm in the MapReduce Framework

Borůvka's algorithm iteratively finds the Minimum Spanning Tree (MST) by identifying the smallest outgoing edge from each component. The MapReduce module is employed to parallelize this process. First, the graph is loaded and divided into smaller chunks, with each mapper working on a portion of the graph to find the minimum edge for its component. To efficiently manage the merging of graph components, a Disjoint Set Union (DSU) data structure should be implemented with union by rank optimization. This helps maintain an efficient structure as components are merged across iterations. In the reduce phase, the identified edges are combined, and the DSU structure is used to update the graph components by merging the connected components. This process is repeated across multiple MapReduce iterations until only a single connected component remains, representing the MST.

4 Implementation

4.1 Algorithm

The solution is based on Borůvka's algorithm, which iteratively identifies the minimum-weight edges connecting different components of the graph and adds them to the MST. The disjoint-set union (DSU) data structure is used to manage the components, ensuring that no cycles are formed. The MapReduce framework is employed to parallelize the selection of minimum-weight edges for each component at every iteration.

Borůvka's Algorithm in MapReduce

```
Input: Graph represented as an adjacency list
Initialize: Disjoint Set Union data structure with rank and parent
while Number of connected components > 1 do
    Map Phase:
        Each mapper processes a portion of the graph to yield outgoing edges for each CC.
    Reduce Phase:
        Reducers collect outgoing edges and find the minimum outgoing edge for each CC.
    Union:
        The minimum edges are used to merge respective components using DSU.
end while
Output: Minimum Spanning Tree (MST) of the graph
```

4.2 Technology

We aim to develop a Go module for the same, specifically using gRPC calls. As suggested by the paper the map task output would be stored in local disk. We are choosing this stack due to our familiarity with gRPC allowing master and workers to communicate efficiently via goroutines provided by Go which utilize available cores efficiently.

5 Evaluation

For testing and evaluating our algorithm, the Florida Sparse Matrix Collection is used, which provides a wide range of real-world sparse graph datasets. The collection includes graphs from social networks, road networks, and scientific computations, making it suitable for testing parallel algorithms.

- Davis, T. A., & Hu, Y. (2011). *The University of Florida Sparse Matrix Collection.* ACM Transactions on Mathematical Software (TOMS), 38(1), 1-25.

5.1 Performance on a specific graph

pesa.mtx is a weighted network graph from the UFL graph dataset.
https://www.cise.ufl.edu/research/sparse/matrices/Gaertner/pesa.html

Table 1: Graph Properties of *pesa.mtx*

Property	**Value**
Nodes	11.7K
Edges	67.8K
Density	0.000984663
Maximum Degree	18
Minimum Degree	4
Average Degree	11
Assortativity	0.205337
Number of Triangles	266.1K
Average Number of Triangles	22
Maximum Number of Triangles	36
Average Clustering Coefficient	0.374996
Maximum k-core	9
Lower Bound of Maximum Clique	5

Figure 1: Graph Visualisation of *pesa.mtx*

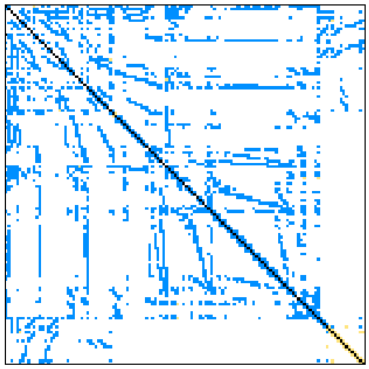


Figure 2: Adjacency matrix of *pesa.mtx*

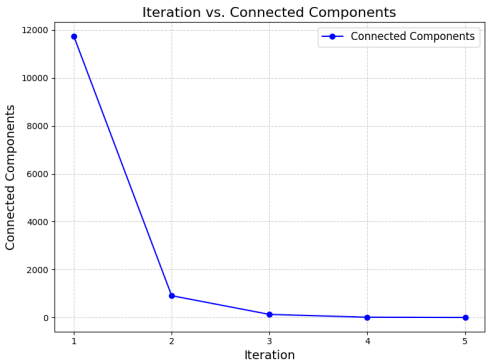


Figure 3: Iterations vs number of connected components in *pesa.mtx*

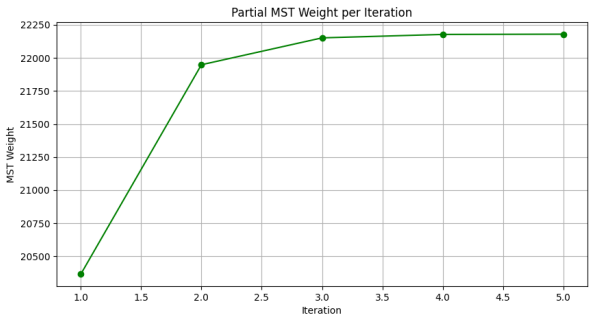


Figure 4: Iteration vs Partial MST weight in *pesa.mtx*

5.2 Overall Performance

File	**MapReduce Total Time (s)**	**Serial Total Time (s)**	**MapReduce Iterations**	**Serial Iterations**	**MST Weight**
email.mtx	0.018960	0.002130	3	3	1132
delaunay_n20.mtx	16.717600	11.500136	7	9	1048575
wing_nodal.mtx	0.305998	0.076856	4	5	10936
19.mtx	0.000325	0.000020	1	1	19
ca-HepTh.mtx	0.120454	0.033665	5	4	9448
sample.mtx	0.000269	0.000024	2	2	217
c-40.mtx	0.148508	0.024462	3	3	4517
roadNet-PA.mtx	11.576190	8.402045	9	10	1087886
Oregon-2.mtx	0.129681	0.026384	4	3	11460
pkustk02.mtx	1.358763	0.320376	5	5	10799
Oregon-1.mtx	0.102578	0.030549	4	4	11173
PGPgiantcompo.mtx	0.108625	0.022855	4	4	10681

Table 2: Performance comparison of MapReduce and Serial implementations of the MST algorithm.

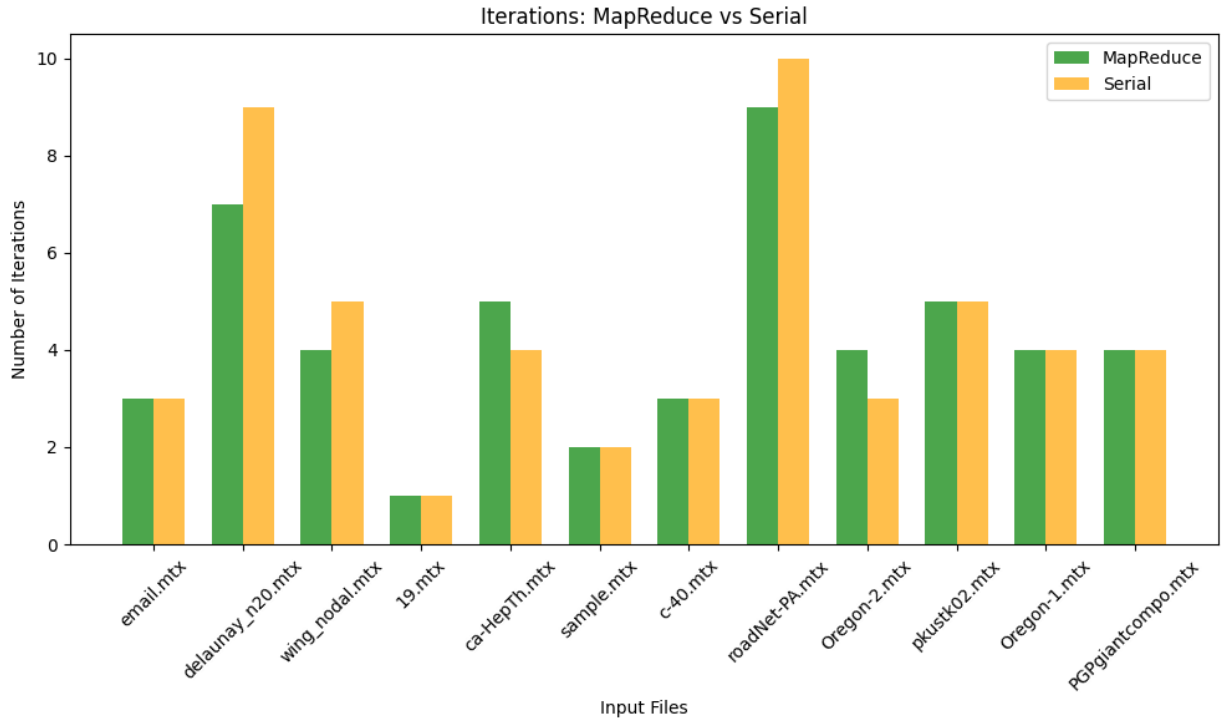


Figure 5: Number of iterations taken by MapReduce and Serial algorithm

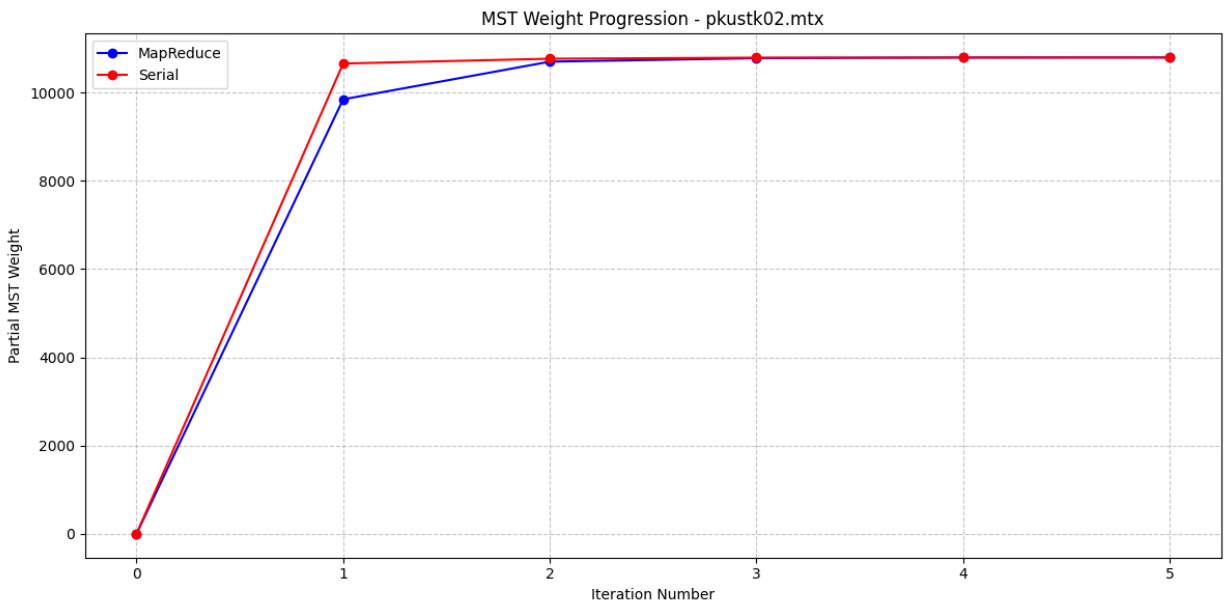
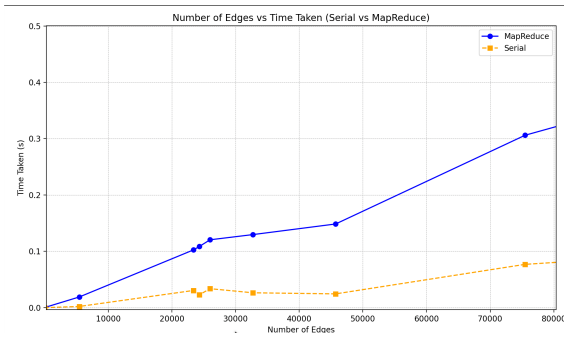
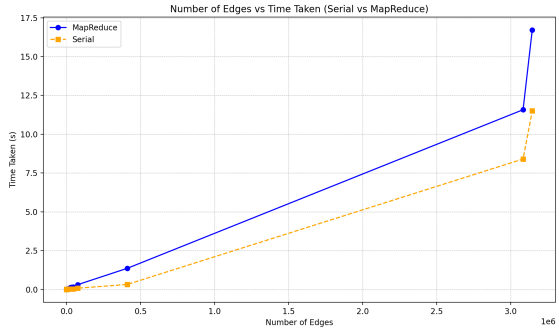


Figure 6: Progression of MST weight calculation in each iteration



(a) Zoomed in at the bottom left



(b) Zoomed out

Figure 7: Number of edges in graph vs Time taken to calculate MST

6 Conclusion

From the results of the evaluation, it is evident that there is no significant advantage of employing map reduce to solve the minimum spanning tree problem. However, there are two key places where there is merit to using map reduce.

- Reading the input graph through map reduce allows for faster processing of graph since lines can be read parallely(creating the adjacency list).
- The process of picking the minimum weighted outgoing edge of each component can be done in parallel, which is supported by the map reduce framework.